STMicroelectronics

AI Accelerators Hands-on HW Design

Jul.2025

life.augmented

LAB 0

Systolic Array for Applying Matrix Multiplication

Instructor:
Ahmed Abdelsalam

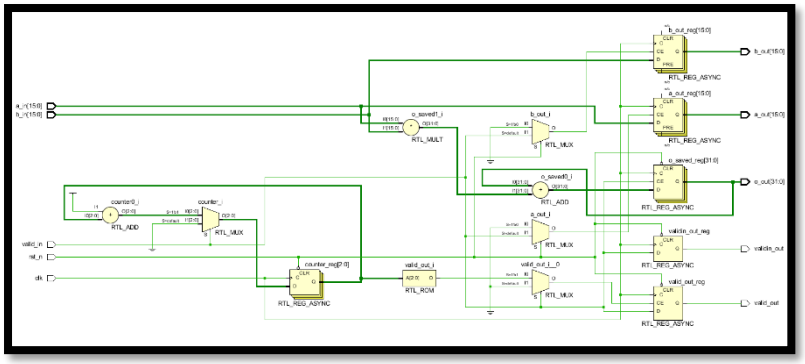
REPORT
Of fully generic Systolic Array for Applying Matrix Multiplication

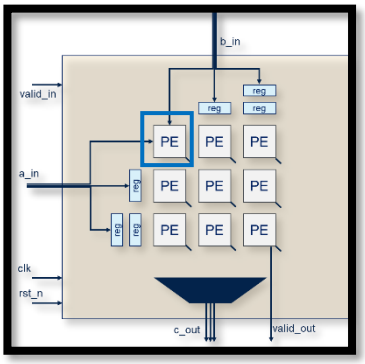
Made by :
George Jan George Shaffik

life.augmented

RTL

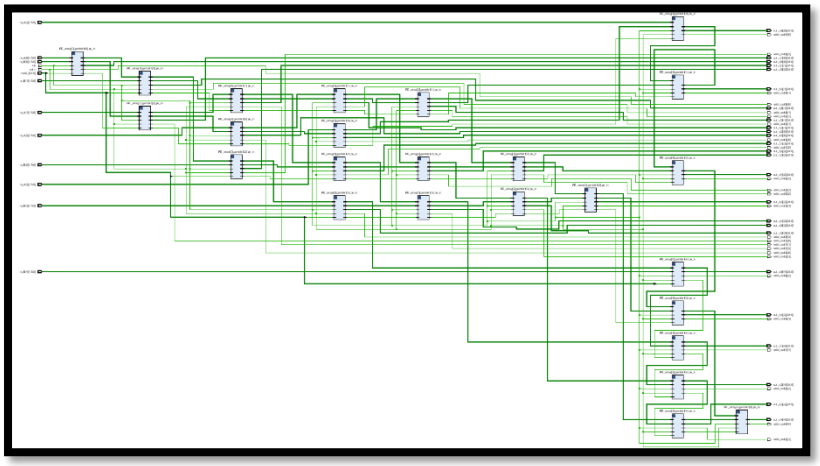
PE (processing element)

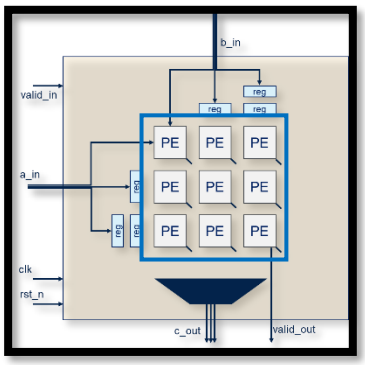




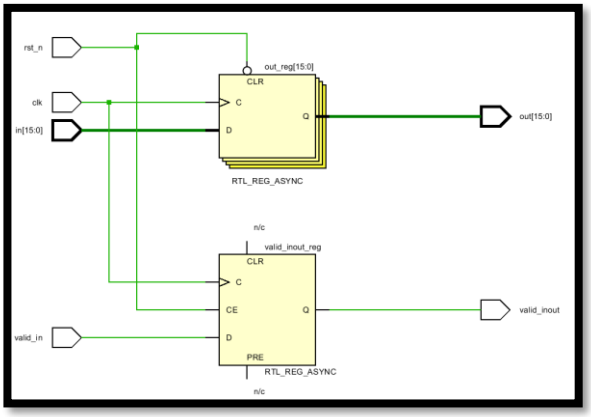
This PE make PE function which is out=out+(in_a * in_b) , transfer valid_in and generate valid_out signal

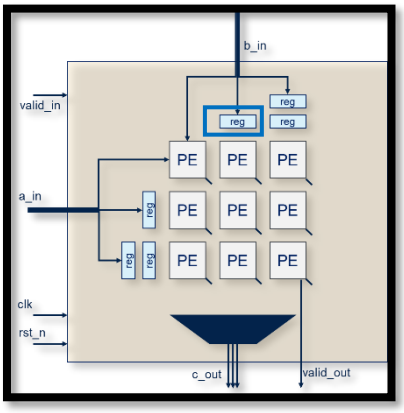
PE array



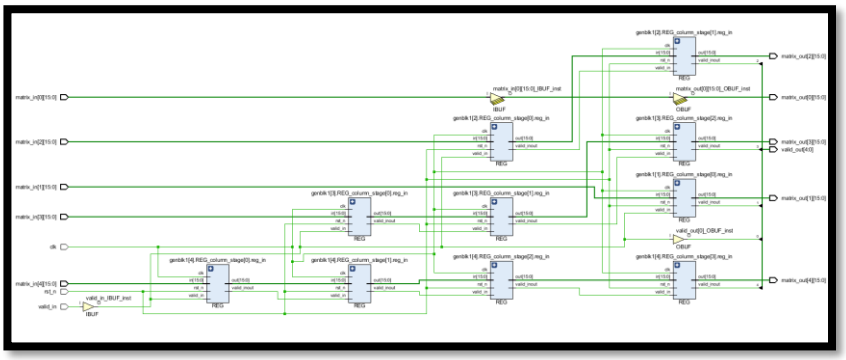


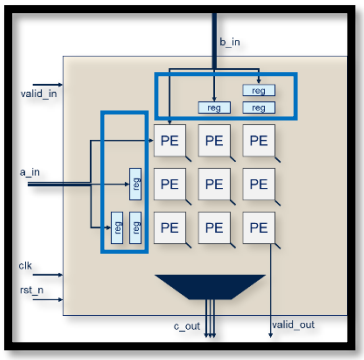
Register



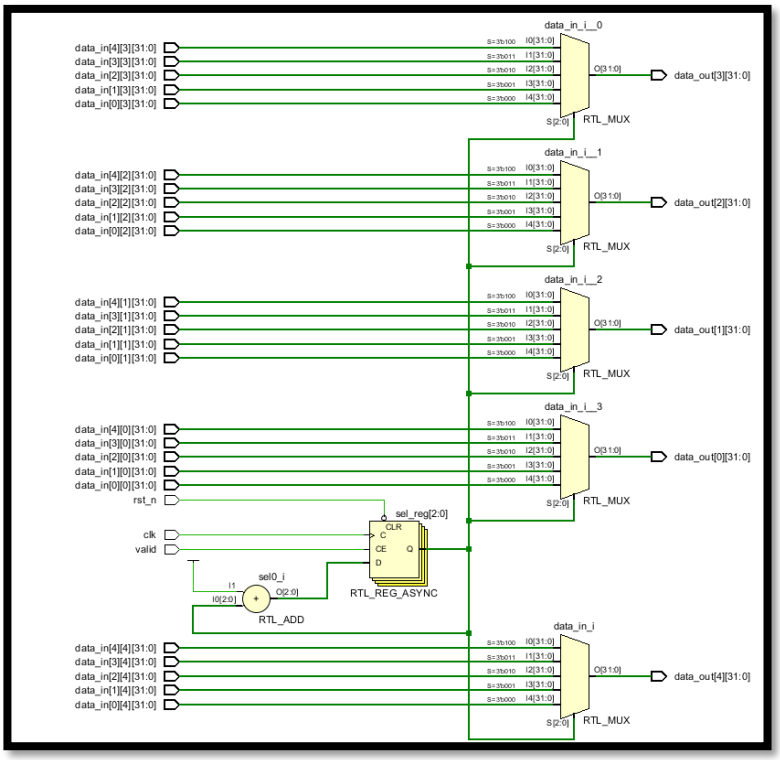


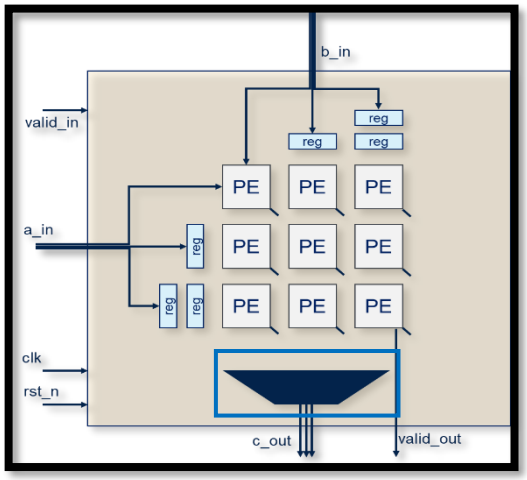
Register array



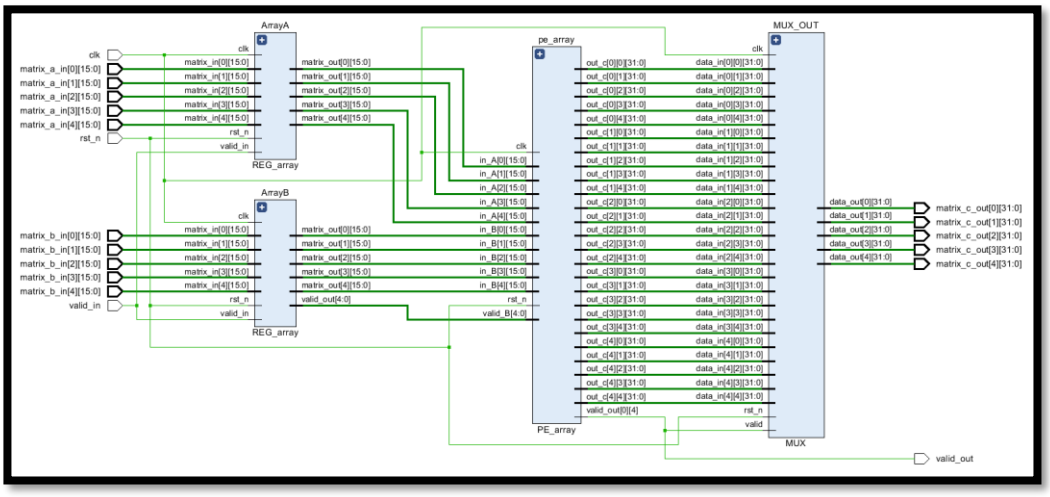


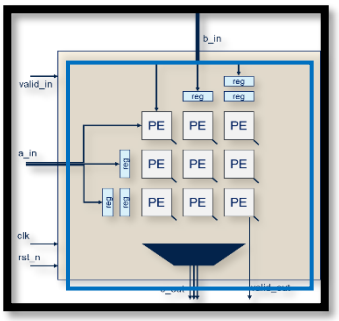
Multiplexer



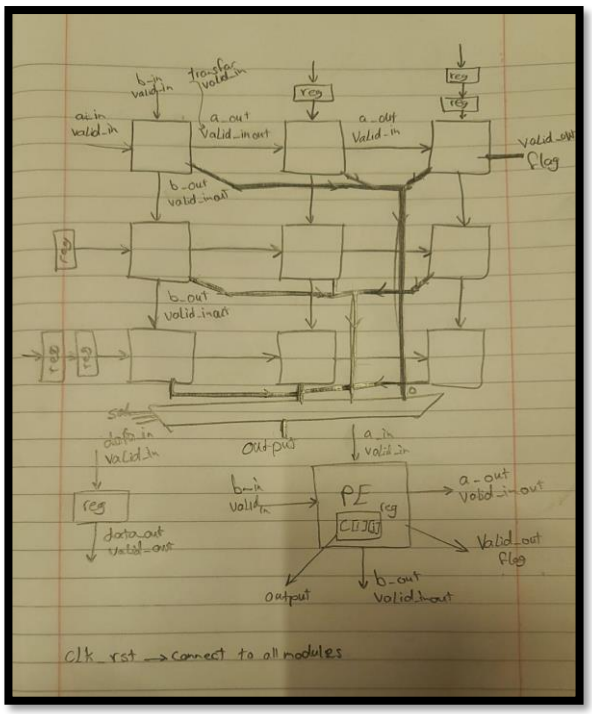


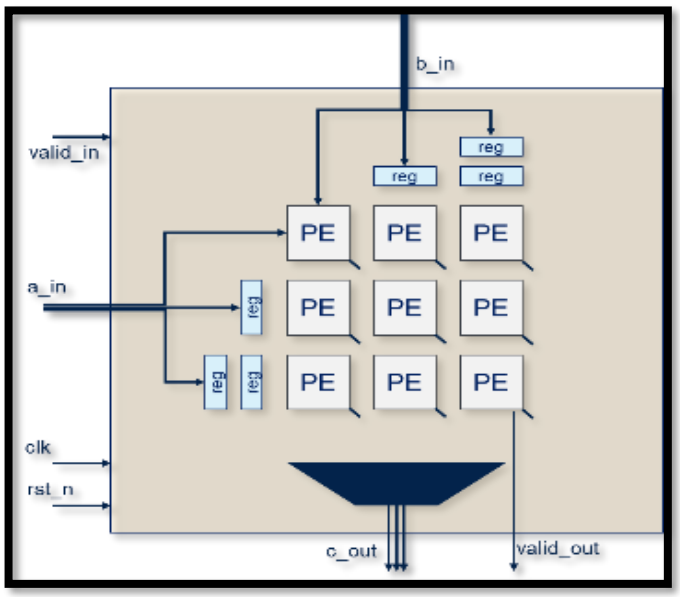
TOP file (Systolic Array)





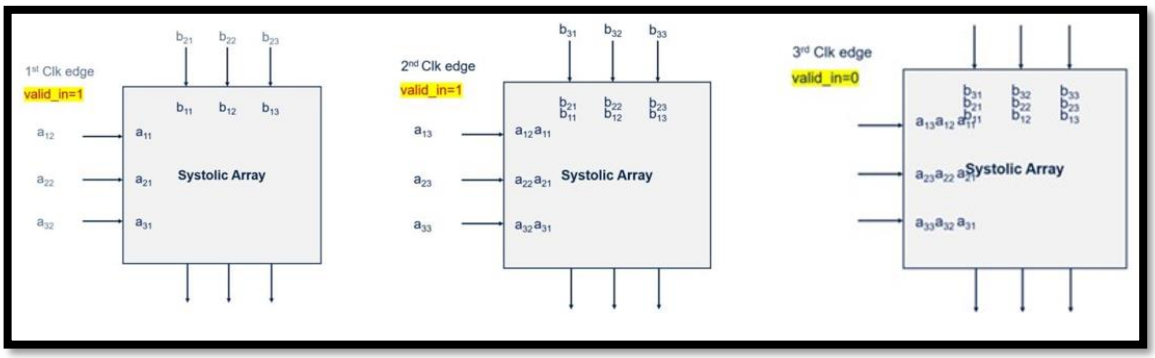
Architecture



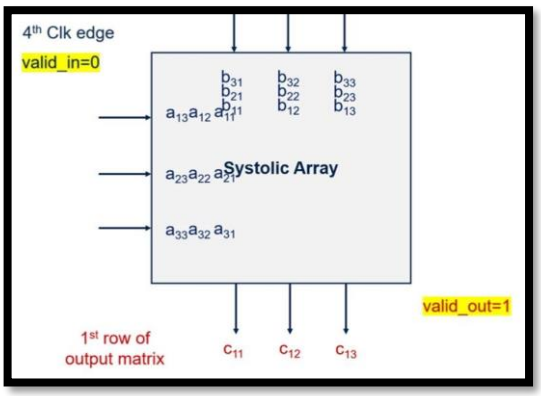


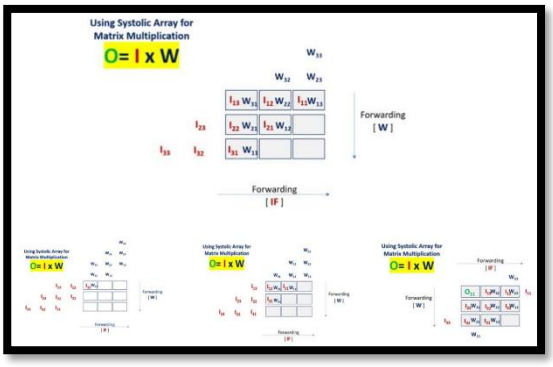
How design work

1. Reset = zero to initialize all elements blocks
2. Valid_in = 1 and add data like photo below then put Valid_in to zero to stop taking inputs



3. Then the output will execute row by row and set Valid_out flag





Simulation Snapshots

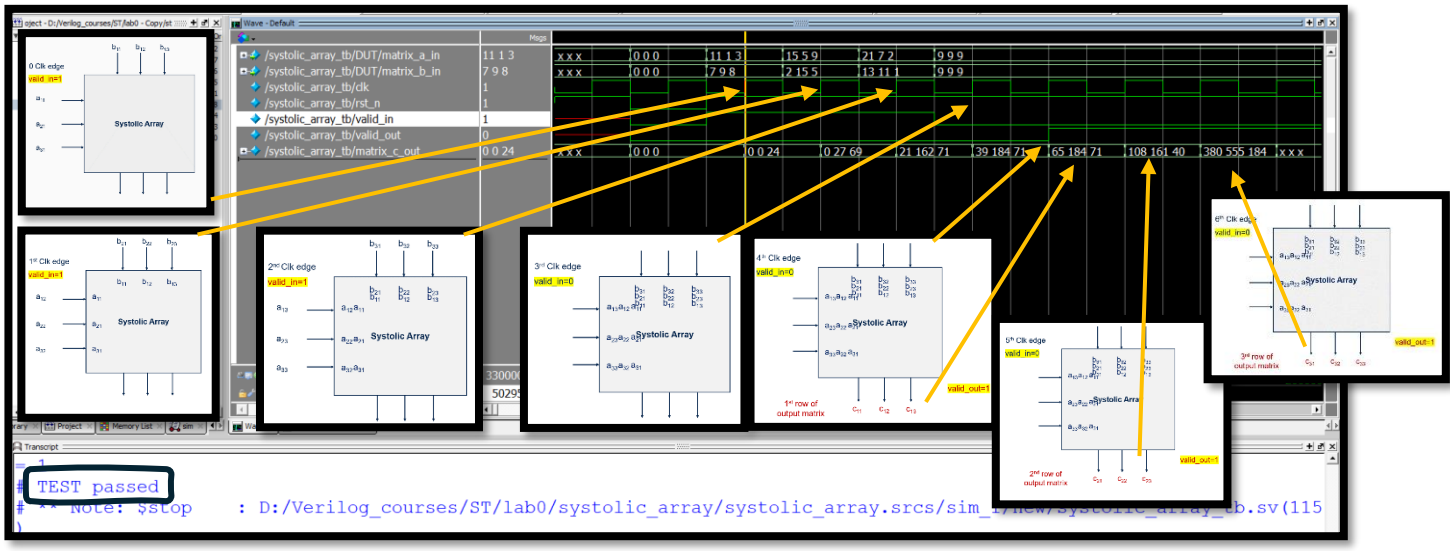
Test with parameter DATAWIDTH=16 and N_SIZE=3

$$\begin{bmatrix} a11 & a12 & a13 \\ a21 & a22 & a23 \\ a31 & a32 & a33 \end{bmatrix} * \begin{bmatrix} b11 & b12 & b13 \\ b21 & b22 & b23 \\ b31 & b32 & b33 \end{bmatrix} = \begin{bmatrix} c11 & c12 & c13 \\ c21 & c22 & c23 \\ c31 & c32 & c33 \end{bmatrix}$$

a11→ **matrix_a_in[0]**
a12 → **matrix_a_in[1]**
a13 → **matrix_a_in[2]**

c11 → **matrix_c_in[0] = 71 ; 40 ; 184**
c12 → **matrix_c_in[1]= 184 ; 161 ; 555**
c13 → **matrix_c_in[2]= 65 ; 108 ; 380**

$$\begin{bmatrix} 3 & 9 & 2 \\ 1 & 5 & 7 \\ 11 & 15 & 21 \end{bmatrix} * \begin{bmatrix} 8 & 9 & 7 \\ 5 & 15 & 2 \\ 1 & 11 & 13 \end{bmatrix} = \begin{bmatrix} 71 & 184 & 65 \\ 40 & 161 & 108 \\ 184 & 555 & 380 \end{bmatrix} = \begin{bmatrix} \mathbf{matrix\_c\_in[0]} & \mathbf{matrix\_c\_in[1]} & \mathbf{matrix\_c\_in[2]} \\ \mathbf{matrix\_c\_in[0]} & \mathbf{matrix\_c\_in[1]} & \mathbf{matrix\_c\_in[2]} \\ \mathbf{matrix\_c\_in[0]} & \mathbf{matrix\_c\_in[1]} & \mathbf{matrix\_c\_in[2]} \end{bmatrix}$$



Testbench code

```
module systolic_array_tb();
    parameter integer DATAWIDTH=16;
    parameter integer N_SIZE=3;
    reg clk,rst_n,valid_in;
    reg [DATAWIDTH-1:0] matrix_a_in [N_SIZE-1:0];
    reg [DATAWIDTH-1:0] matrix_b_in [N_SIZE-1:0];
    wire valid_out;
    wire [2*(DATAWIDTH)-1:0] matrix_c_out [N_SIZE-1:0];

//DUT
systolic_array # (.DATAWIDTH(DATAWIDTH),.N_SIZE(N_SIZE)) DUT
(
    .clk (clk),.rst_n (rst_n),.valid_in (valid_in),
    .matrix_a_in (matrix_a_in),
    .matrix_b_in (matrix_b_in),
    .valid_out (valid_out),
    .matrix_c_out (matrix_c_out)
);
```

```
// clk
    initial
    begin
        clk=0;
    forever
        #100 clk=~clk;
    end
// test inputs
initial
begin
        rst_n=1;
    @(negedge clk)
        rst_n=0;
        valid_in=0;
        matrix_a_in = '{default:0};
        matrix_b_in = '{default:0};
    @(negedge clk)
        rst_n=1;
        valid_in=1;
        matrix_a_in[0]='d3;
        matrix_a_in[1]='d1;
        matrix_a_in[2]='d11;

        matrix_b_in[0]='d8;
        matrix_b_in[1]='d9;
        matrix_b_in[2]='d7;
    @(negedge clk)
            matrix_a_in[0]='d9;
            matrix_a_in[1]='d5;
            matrix_a_in[2]='d15;

            matrix_b_in[0]='d5;
            matrix_b_in[1]='d15;
            matrix_b_in[2]='d2;
    @(negedge clk)
            matrix_a_in[0]='d2;
            matrix_a_in[1]='d7;
            matrix_a_in[2]='d21;

            matrix_b_in[0]='d1;
            matrix_b_in[1]='d11;
            matrix_b_in[2]='d13;
    @(negedge clk)
        valid_in=0;
            matrix_a_in[0]='d9;
            matrix_a_in[1]='d9;
            matrix_a_in[2]='d9;

            matrix_b_in[0]='d9;
            matrix_b_in[1]='d9;
            matrix_b_in[2]='d9;
    @(negedge clk)
    @(negedge clk)
    if (matrix_c_out[0]!=71 || matrix_c_out[1]!=184 || matrix_c_out[2]!=65)
    begin
        $display ("ERROR");
        $stop;
    end
    @(negedge clk)
    if (matrix_c_out[0]!=40 || matrix_c_out[1]!=161 || matrix_c_out[2]!=108)
    begin
        $display ("ERROR");
        $stop;
    end
    @(negedge clk)
    if (matrix_c_out[0]!=184 || matrix_c_out[1]!=555 || matrix_c_out[2]!=380)
    begin
        $display ("ERROR");
        $stop;
    end
$display ("TEST passed");
    #1500;
$stop;
end

// monitoring
initial
    $monitor ("reset = %b,Valid input = %b ,input A = %p,input B = %p,output = %p,Valid output = %b"
    , rst_n , valid_in , matrix_a_in , matrix_b_in , matrix_c_out , valid_out);
endmodule
```

Self checking

Another testbench code that write on file.log

```
23  module systolic_array_tb();
24      // Parameters
25      parameter integer DATAWIDTH = 16;
26      parameter integer N_SIZE = 3;
27
28      // Signals
29      reg clk, rst_n, valid_in;
30      reg [DATAWIDTH-1:0] matrix_a_in [N_SIZE-1:0];
31      reg [DATAWIDTH-1:0] matrix_b_in [N_SIZE-1:0];
32      wire valid_out;
33      wire [2*(DATAWIDTH)-1:0] matrix_c_out [N_SIZE-1:0];
34
35      // File handle for logging
36      integer file_handle;
37
38      // Stored input matrices for logging
39      reg [DATAWIDTH-1:0] matrix_a_log [N_SIZE-1:0][N_SIZE-1:0];
40      reg [DATAWIDTH-1:0] matrix_b_log [N_SIZE-1:0][N_SIZE-1:0];
41
42      // Instantiate the Design Under Test (DUT)
43      systolic_array # (.DATAWIDTH(DATAWIDTH), .N_SIZE(N_SIZE)) DUT (
44          .clk(clk),
```

```
45          .rst_n(rst_n),
46          .valid_in(valid_in),
47          .matrix_a_in(matrix_a_in),
48          .matrix_b_in(matrix_b_in),
49          .valid_out(valid_out),
50          .matrix_c_out(matrix_c_out)
51      );
52
53      // Clock generator
54      initial begin
55          clk = 0;
56          forever #50 clk = ~clk; // 100ns period -> 10MHz clock
57      end
58
59      // Test sequence
60      initial begin
61          // Open the log file for writing. This will create 'file.log'.
62          file_handle = $fopen("file.log", "w");
63          if (file_handle == 0) begin
64              $display("ERROR: Could not open file.log for writing.");
65              $finish;
66          end
```

```
67
68          // --- Reset and Initialization ---
69          rst_n = 1;
70          @(negedge clk);
71          rst_n = 0;
72          valid_in = 0;
73          matrix_a_in = '{default:0};
74          matrix_b_in = '{default:0};
75          @(negedge clk);
76          rst_n = 1;
77
78          // --- Apply Inputs and Log Them ---
79          $fdisplay(file_handle, "--- Simulation Log ---");
80          $fdisplay(file_handle, "Input Matrix A:");
81
82          // Cycle 1
83          @(negedge clk);
84          valid_in = 1;
85          matrix_a_in[0] = 3; matrix_a_in[1] = 1; matrix_a_in[2] = 11;
86          matrix_b_in[0] = 8; matrix_b_in[1] = 9; matrix_b_in[2] = 7;
87          // Store for logging
88          matrix_a_log[0] = matrix_a_in;
```

```
89          matrix_b_log[0] = matrix_b_in;
90
91          // Cycle 2
92          @(negedge clk);
93          matrix_a_in[0] = 9; matrix_a_in[1] = 5; matrix_a_in[2] = 15;
94          matrix_b_in[0] = 5; matrix_b_in[1] = 15; matrix_b_in[2] = 2;
95          // Store for logging
96          matrix_a_log[1] = matrix_a_in;
97          matrix_b_log[1] = matrix_b_in;
98
99          // Cycle 3
100         @(negedge clk);
101         matrix_a_in[0] = 2; matrix_a_in[1] = 7; matrix_a_in[2] = 21;
102         matrix_b_in[0] = 1; matrix_b_in[1] = 11; matrix_b_in[2] = 13;
103         // Store for logging
104         matrix_a_log[2] = matrix_a_in;
105         matrix_b_log[2] = matrix_b_in;
106
107         // De-assert valid_in
108         @(negedge clk);
109         valid_in = 0;
110         matrix_a_in = '{default:0};
```

```
111         matrix_b_in = '{default:0};
112
113         // --- Print stored input matrices to the log file ---
114         // Note: Systolic arrays process rows/columns. How you print depends on how you fed them.
115         // This assumes matrix_a_in was a column and matrix_b_in was a row each cycle.
116         $fdisplay(file_handle, "Matrix A (fed as columns):");
117         for (int i = 0; i < N_SIZE; i++) begin
118             $fdisplay(file_handle, "\t%d\t%d\t%d", matrix_a_log[0][i], matrix_a_log[1][i], matrix_a_lo
119         end
120
121         $fdisplay(file_handle, "\nInput Matrix B (fed as rows):");
122         for (int i = 0; i < N_SIZE; i++) begin
123             $fdisplay(file_handle, "\t%d\t%d\t%d", matrix_b_log[i][0], matrix_b_log[i][1], matrix_b_lo
124         end
125
126         $fdisplay(file_handle, "\n----------------------");
127         $fdisplay(file_handle, "Output Matrix C:");
128
129         // Wait for all valid outputs
130         repeat (N_SIZE) begin
131             @(posedge valid_out);
132             @(negedge clk); // Ensure data is stable
```

```
133             $fdisplay(file_handle, "\t%p", matrix_c_out);
134         end
135
136         // --- End Simulation ---
137         $display("TEST PASSED: All outputs received and logged.");
138         #100;
139
140         // Close the log file
141         $fclose(file_handle);
142         $finish;
143     end
144
145     // Console monitoring (optional, but good for live debugging)
146     initial
147         $monitor("Time=%0t: rst_n=%b, valid_in=%b, valid_out=%b, matrix_c_out=%p",
148                  $time, rst_n, valid_in, valid_out, matrix_c_out);
149
150 endmodule
```

file.log - Notepad

File Edit Format View Help

```
--- Simulation Log ---
Input Matrix A:
Matrix A (fed as columns):
            3       9       2
            1       5       7
           11      15      21

Input Matrix B (fed as rows):
            8       9       7
            5      15       2
            1      11      13

----------------------
Output Matrix C:
        '{65,184,71}
```

file (2).log - Notepad

File Edit Format View Help

```
Vivado Simulator 2018.2
Time resolution is 1 ps
Time=0: rst_n=1, valid_in=x, valid_out=x, matrix_c_out='{32'bxxxxxxxxxxxxxxxx
Time=100000: rst_n=0, valid_in=0, valid_out=0, matrix_c_out='{0,0,0}
Time=200000: rst_n=1, valid_in=0, valid_out=0, matrix_c_out='{0,0,0}
Time=300000: rst_n=1, valid_in=1, valid_out=0, matrix_c_out='{0,0,0}
Time=350000: rst_n=1, valid_in=1, valid_out=0, matrix_c_out='{0,0,24}
Time=450000: rst_n=1, valid_in=1, valid_out=0, matrix_c_out='{0,27,69}
Time=550000: rst_n=1, valid_in=1, valid_out=0, matrix_c_out='{21,162,71}
Time=600000: rst_n=1, valid_in=0, valid_out=0, matrix_c_out='{21,162,71}
Time=650000: rst_n=1, valid_in=0, valid_out=0, matrix_c_out='{39,184,71}
Time=750000: rst_n=1, valid_in=0, valid_out=1, matrix_c_out='{65,184,71}
Time=850000: rst_n=1, valid_in=0, valid_out=1, matrix_c_out='{108,161,40}
Time=950000: rst_n=1, valid_in=0, valid_out=1, matrix_c_out='{380,555,184}
```

Design code

PE (processing element)

- This module has reset input use to initialized o_saved (register that accumulate)
- Make the main function of PE which is o_saved =o_saved + a_in*b_in
- Transfer a_in to PE on its right side
- Transfer b_in to PE below it
- Transfer valid_in throw output (validin_out)
- Used valid_in to stop accumulate the multiplication when valid_in ==0
- Execute valid_out = 1 when PE make its function N_SIZE times

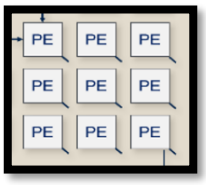
```
module PE # (parameter integer DATAWIDTH = 16, parameter integer N_SIZE =5)  (
    input clk , rst_n , valid_in ,
    input [DATAWIDTH-1:0] a_in , b_in ,
    output reg [DATAWIDTH-1:0] a_out , b_out ,
    output [2*(DATAWIDTH)-1:0] o_out ,
    output reg valid_out,validin_out
    );

reg [2*(DATAWIDTH)-1:0] o_saved;
reg [$clog2(N_SIZE)-1:0] counter;
assign o_out = o_saved;
always @ (posedge clk or negedge rst_n)
begin
    if (rst_n == 1'b0)
    begin
        o_saved <= 0;
        valid_out <= 1'b0;
        counter <= 0;
        validin_out<=1'b0;
    end
    else if (valid_in == 1'b1)
    begin
```

```
        a_out <= a_in;
        b_out <= b_in;
        o_saved <= o_saved + a_in*b_in;
        counter <= counter + 1;
        validin_out<=valid_in;  //transfar valid_in
        if (counter == N_SIZE-1)//generate valid_out flag
            valid_out <= 1'b1;
    end
    else if (valid_in == 1'b0)
    begin
        counter <= 0;
        o_saved <= o_saved;
        validin_out <= valid_in;
    end
    else
        o_saved <= o_saved;
end
endmodule
```

PE array

- this module used to connect PE as the picture but with any value of DATAWIDTH, N_SIZE
- this module make a generic PE array




```
module PE_array # (parameter integer DATAWIDTH=16,parameter integer N_SIZE=5)(
    input [DATAWIDTH-1:0] in_A [N_SIZE-1:0],
    input [DATAWIDTH-1:0] in_B [N_SIZE-1:0],
    input [N_SIZE-1:0]valid_B ,
    input clk,rst_n,
    output [2*(DATAWIDTH)-1:0] out_c [N_SIZE-1:0][N_SIZE-1:0],
    output valid_out [N_SIZE-1:0][N_SIZE-1:0]
    );
wire [DATAWIDTH-1:0] A [N_SIZE-1:0][N_SIZE-1:0];
wire [DATAWIDTH-1:0] B [N_SIZE-1:0][N_SIZE-1:0];
reg valid [N_SIZE-1:0][N_SIZE-1:0];
generate
genvar i,j,k;
for (k=0 ; k< N_SIZE ; k=k+1)
begin
    assign A [k][0]=in_A[k];
    assign B [0][k]=in_B[k];
    assign valid [0][k]= valid_B[k];
end
for (i=0 ; i<N_SIZE ; i=i+1)
begin : PE_array
```

```

    for (j=0 ; j<N_SIZE ; j=j+1)
    begin

    PE # (.DATAWIDTH (DATAWIDTH),.N_SIZE (N_SIZE)) pe_in (
    .clk (clk), .rst_n (rst_n), .valid_in (valid [i][j]),
    .a_in (A[i][j]), .b_in (B[i][j]),
    .a_out (A[i][j+1]), .b_out (B[i+1][j]),
    .o_out (out_c[i][j]),
    .valid_out (valid_out[i][j]),.validin_out(valid [i+1][j])
    );
    end
end
endgenerate
endmodule
```

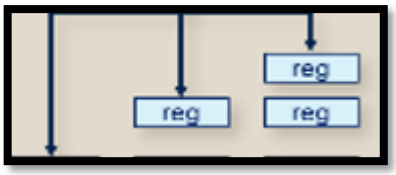
Register

- this generic Register used to take the input (data(in) , valid_in) in clock and delay it from entering the design by 1 clock cycle
- it has reset input to initialized the output by 0

```
module REG # (parameter integer DATAWIDTH = 16)  (
    input clk ,valid_in , rst_n,
    input [DATAWIDTH-1:0] in ,
    output reg [DATAWIDTH-1:0] out ,
    output reg valid_inout
    );
always @ (posedge clk or negedge rst_n)
begin
    if(rst_n==0)
        out<=0;
    else
    begin
    out<=in;
    valid_inout<=valid_in;
    end
end
endmodule
```

Register array

- this generic module use to generate the corresponding array of register but for any value of DATAWIDTH , N_SIZE




```
module REG_array # (parameter integer DATAWIDTH=16,parameter integer N_SIZE=5)
(
    input clk,valid_in,rst_n,
    input [DATAWIDTH-1:0] matrix_in [N_SIZE-1:0],
    output [N_SIZE-1:0]valid_out,
    output [DATAWIDTH-1:0] matrix_out [N_SIZE-1:0]
);
wire [DATAWIDTH-1:0] A [N_SIZE-1:0][N_SIZE-1:0];
wire valid_c [N_SIZE-1:0][N_SIZE-1:0] ;

generate
genvar i,k;
for (i=0 ; i<N_SIZE ; i=i+1)
begin
    assign A[i][0] = matrix_in [i];
    assign valid_c [i][0] = valid_in;
    if (i==0)
    begin
            assign matrix_out[i] = matrix_in [0];
            assign valid_out [i]= valid_in;
    end
```

```
    else
    begin
        for (k=0 ; k<i ; k=k+1)
        begin : REG_column_stage

        REG # (.DATAWIDTH(DATAWIDTH))  reg_in(
                    .clk (clk),.valid_in (valid_c [i][k]),
                    .rst_n(rst_n),
                    .in (A[i][k]),
                    .out (A[i][k+1]),
                    .valid_inout (valid_c [i][k+1])
                    );
        if (k+1==i)
        begin
            assign matrix_out[i] = A[i][k+1];
            assign valid_out [i]= valid_c [i][k+1];
        end
        end
    end
end
endgenerate
endmodule
```

Multiplexer

- This generic mux select which row will be the output at this clock
- It is generic for DATAWIDTH ,N_SIZE

```
module MUX #(parameter integer DATAWIDTH= 16,parameter integer N_SIZE= 5)
(   input valid , clk ,rst_n,
    input  [2*(DATAWIDTH)-1:0] data_in [N_SIZE-1:0][N_SIZE-1:0],
    output [2*(DATAWIDTH)-1:0] data_out[N_SIZE-1:0]
);
reg [$clog2(N_SIZE)-1:0] sel;
    always @ (posedge clk or negedge rst_n)
    begin
    if (rst_n==1'b0)
        sel<=0;
    else if (valid==1'b1)
    begin
        sel<=sel+1 ;
    end
    end
    assign data_out = data_in[sel] ;
endmodule
```

TOP file (Systolic Array)

- This generic module instantiate all the above module to generate a generic Systolic Array With any value of DATAWIDTH , N_SIZE

```
module systolic_array # (parameter integer DATAWIDTH=16,parameter integer N_SIZE=5)
(
    input clk,rst_n,valid_in,
    input [DATAWIDTH-1:0] matrix_a_in [N_SIZE-1:0],
    input [DATAWIDTH-1:0] matrix_b_in [N_SIZE-1:0],
    output valid_out,
    output [2*(DATAWIDTH)-1:0] matrix_c_out [N_SIZE-1:0]
);
//A reg array
wire [DATAWIDTH-1:0] output_A [N_SIZE-1:0];
wire [N_SIZE-1:0]valid_A,rst_A ;
wire [2*(DATAWIDTH)-1:0] out_c [N_SIZE-1:0][N_SIZE-1:0];
wire valid_out_matrix [N_SIZE-1:0][N_SIZE-1:0];
wire Start_out [N_SIZE-1:0][N_SIZE-1:0];

REG_array # (.DATAWIDTH(DATAWIDTH),.N_SIZE(N_SIZE)) ArrayA
(
    .clk (clk),.valid_in (valid_in),.rst_n(rst_n),
    .matrix_in (matrix_a_in),
    .valid_out(valid_A),
    .matrix_out (output_A)
);
```

```
//B reg array
wire [DATAWIDTH-1:0] output_B [N_SIZE-1:0];
wire [N_SIZE-1:0]valid_B,rst_B ;

REG_array # (.DATAWIDTH(DATAWIDTH),.N_SIZE(N_SIZE)) ArrayB
(
    .clk (clk),.valid_in (valid_in),.rst_n(rst_n),
    .matrix_in (matrix_b_in),
    .valid_out(valid_B),
    .matrix_out (output_B)
);

//PE_array
PE_array # (.DATAWIDTH(DATAWIDTH),.N_SIZE(N_SIZE)) pe_array (
    .in_A (output_A),
    .in_B (output_B),
    .valid_B (valid_B),
    .clk(clk),.rst_n(rst_n),
    .out_c (out_c),
    .valid_out (valid_out_matrix)
    );
```

```
//valid flag
wire Start_out_flag;
assign valid_out = valid_out_matrix[0][N_SIZE-1];
assign Start_out_flag = valid_out_matrix[0][N_SIZE-2];

//output
MUX #(.DATAWIDTH(DATAWIDTH),.N_SIZE(N_SIZE)) MUX_OUT
(   .valid (valid_out), .clk (clk),.rst_n(rst_n),
    .data_in (out_c),
    .data_out(matrix_c_out)
);
endmodule
```